

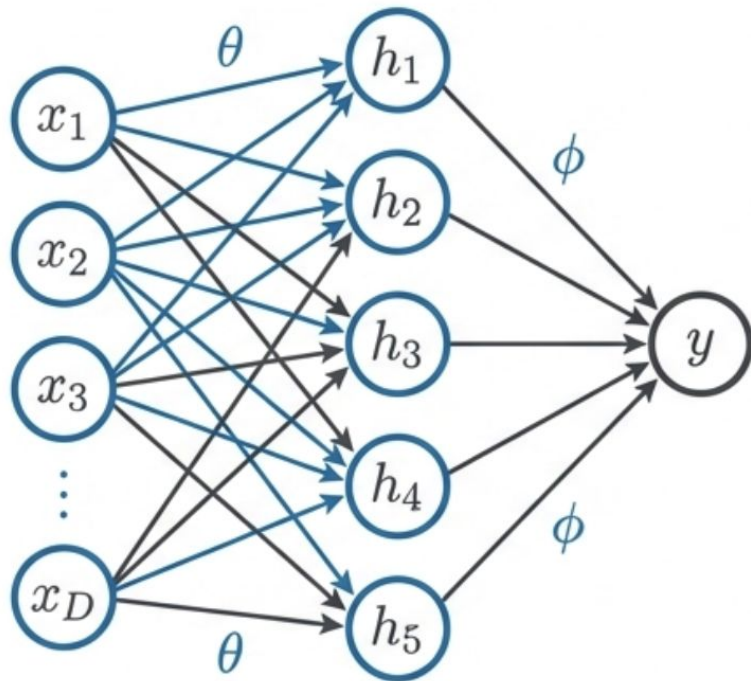
Join the lecture online on your dashboard.

Deep Network Foundations

Mathematical Intuition & Architecture

Recap : Shallow Networks

The Shallow Network Structure



The Shallow Definition:

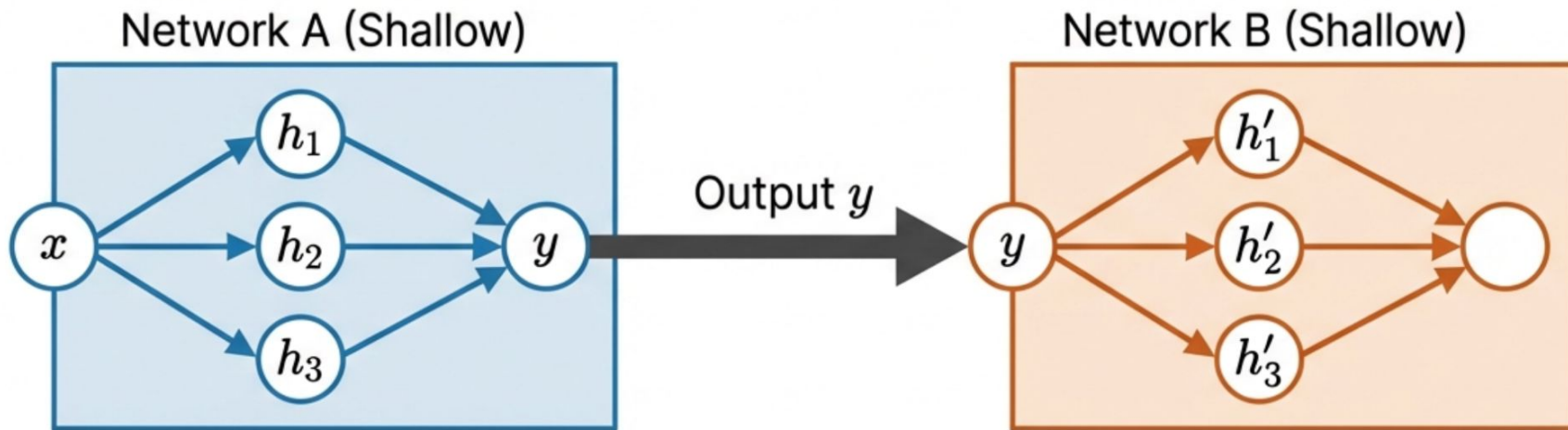
$$y = \phi_0 + \sum_{j=1}^D \phi_j h_j$$

where

$$h_j = \text{ReLU}(\theta_{j0} + \theta_{j1}x)$$

Deep NNs: Increasing Hidden Layers

Understanding Composition



Mathematical Chaining:

Network A: $y = \phi_0 + \sum \phi_j h_j$

Network B: $h'_j = \text{ReLU}(\theta'_{j0} + \theta'_{j1} y)$

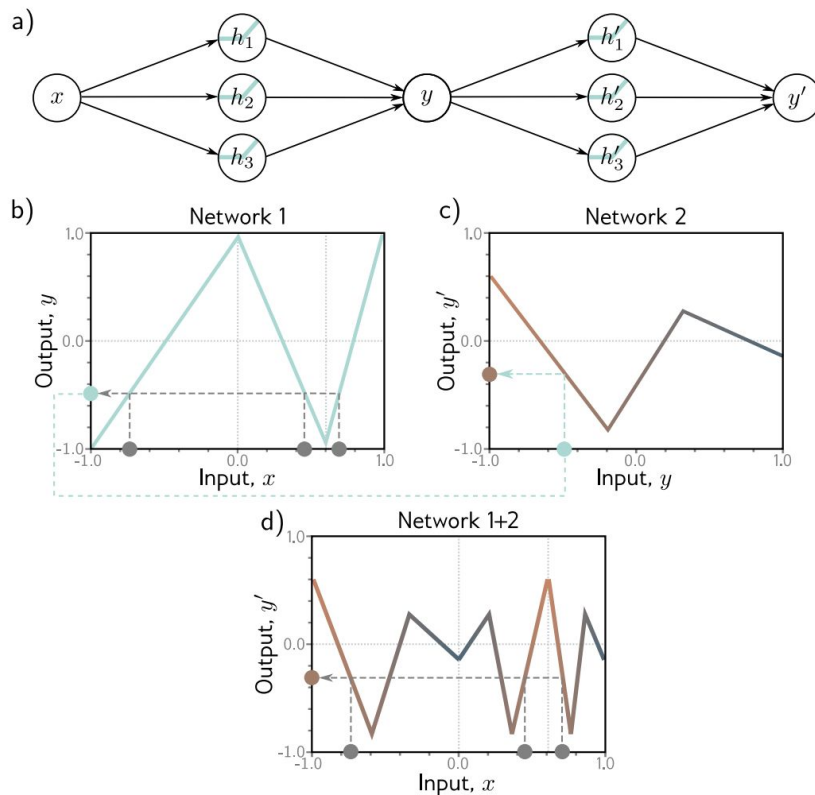
From Shallow to Deep Networks

A way to think about composing networks is that the first network “folds” the input space x back onto itself.

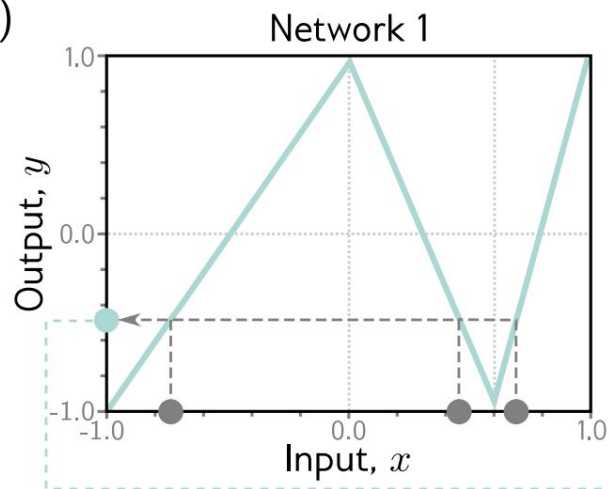
Thus multiple inputs generate the same output.

Then the second network applies a function, which is replicated at all points that were folded on top of one another.

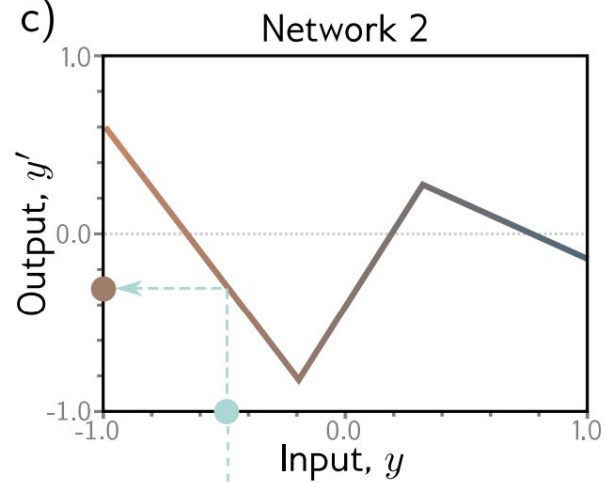
This allows us to create complex distributions using a combination of layers.



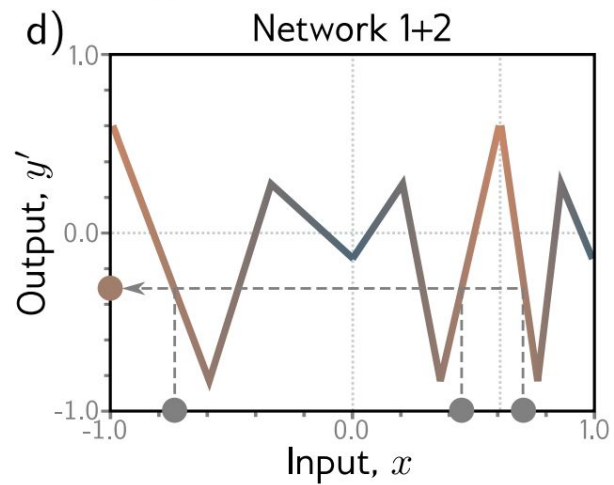
b)



c)



d)



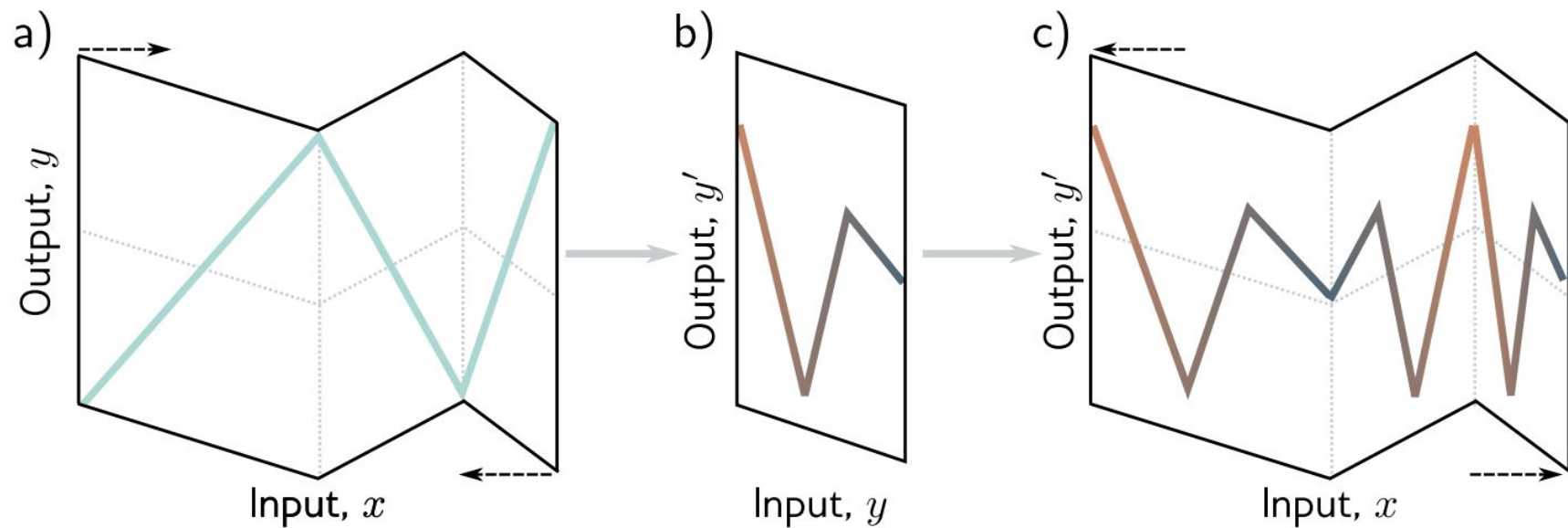
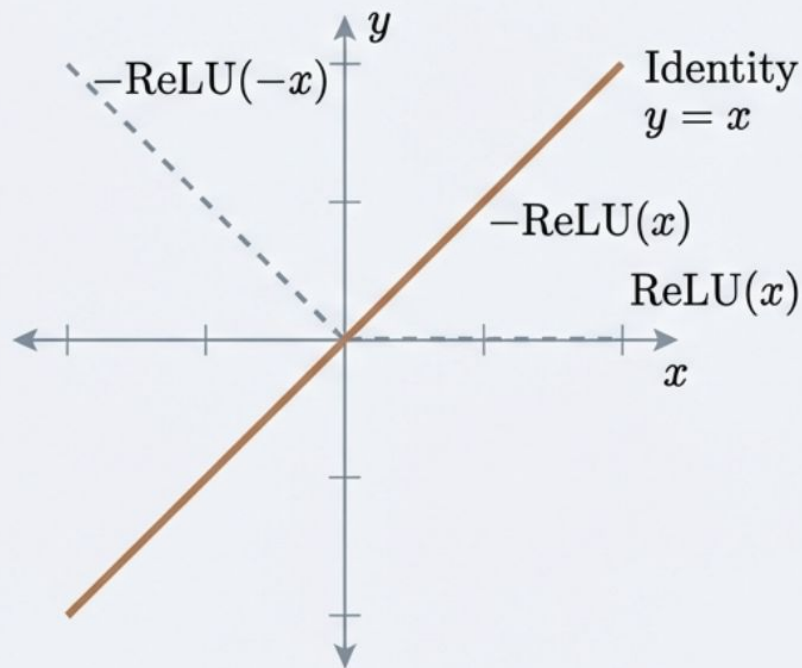


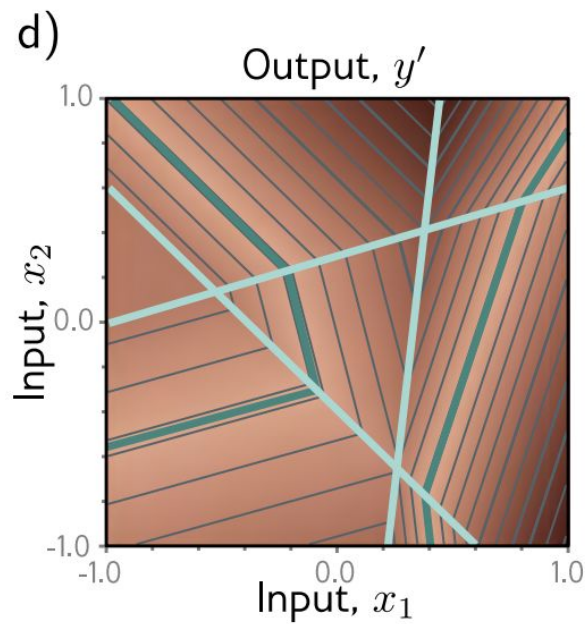
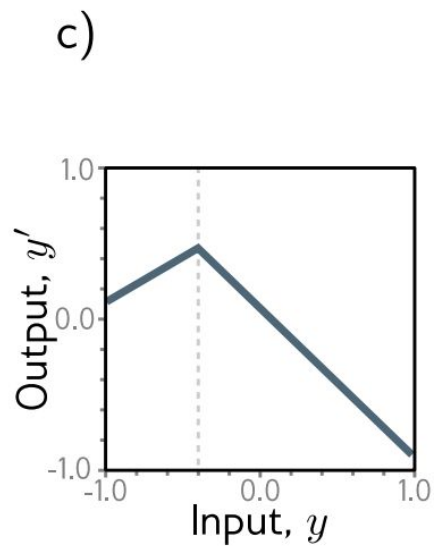
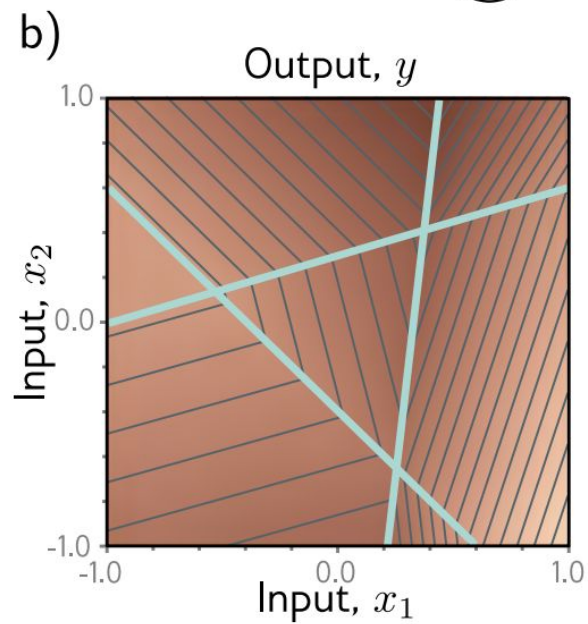
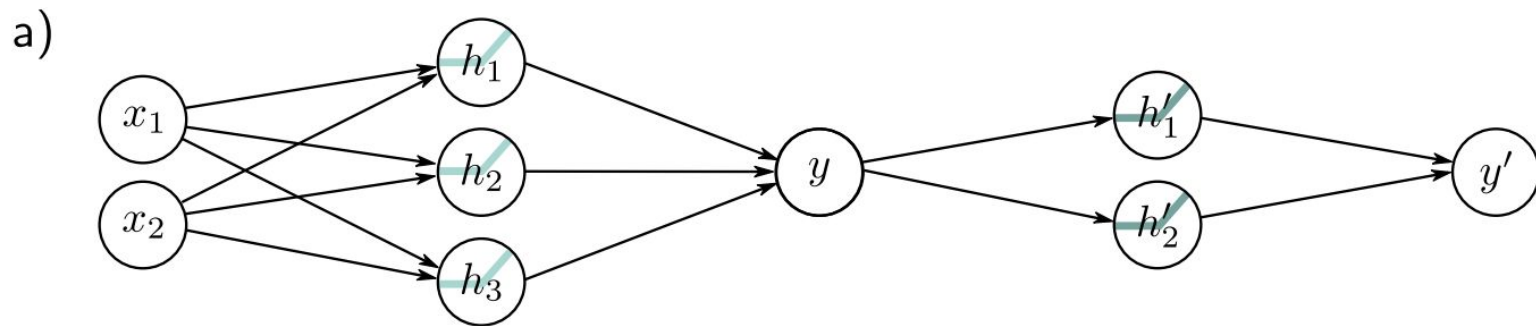
Figure 4.3 Deep networks as folding input space. a) One way to think about the first network from figure 4.1 is that it “folds” the input space back on top of itself. b) The second network applies its function to the folded space. c) The final output is revealed by “unfolding” again.

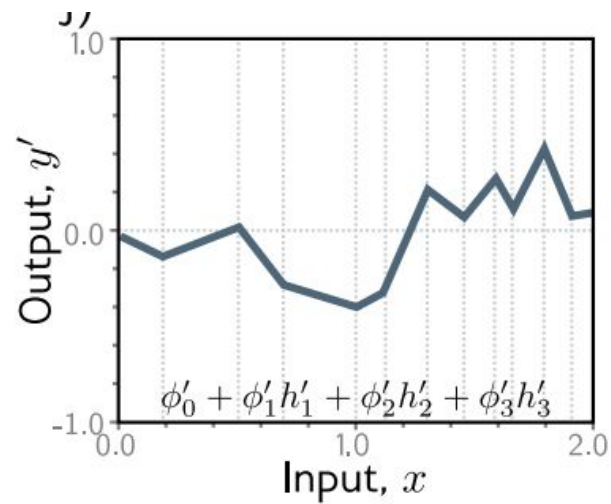
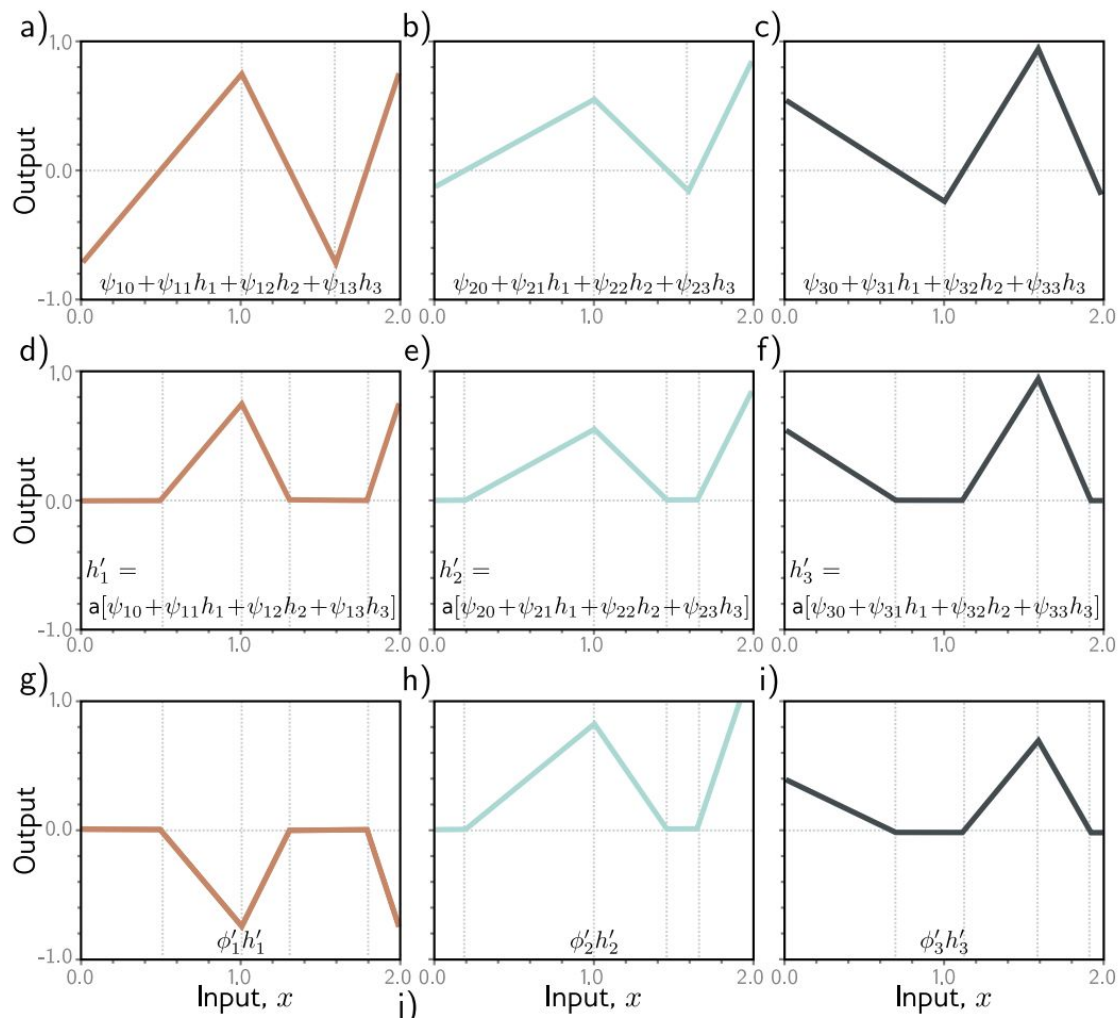
Problem 4.7: Constructing the Identity Function

Choose parameters for a shallow ReLU network to create the identity function $y = x$.

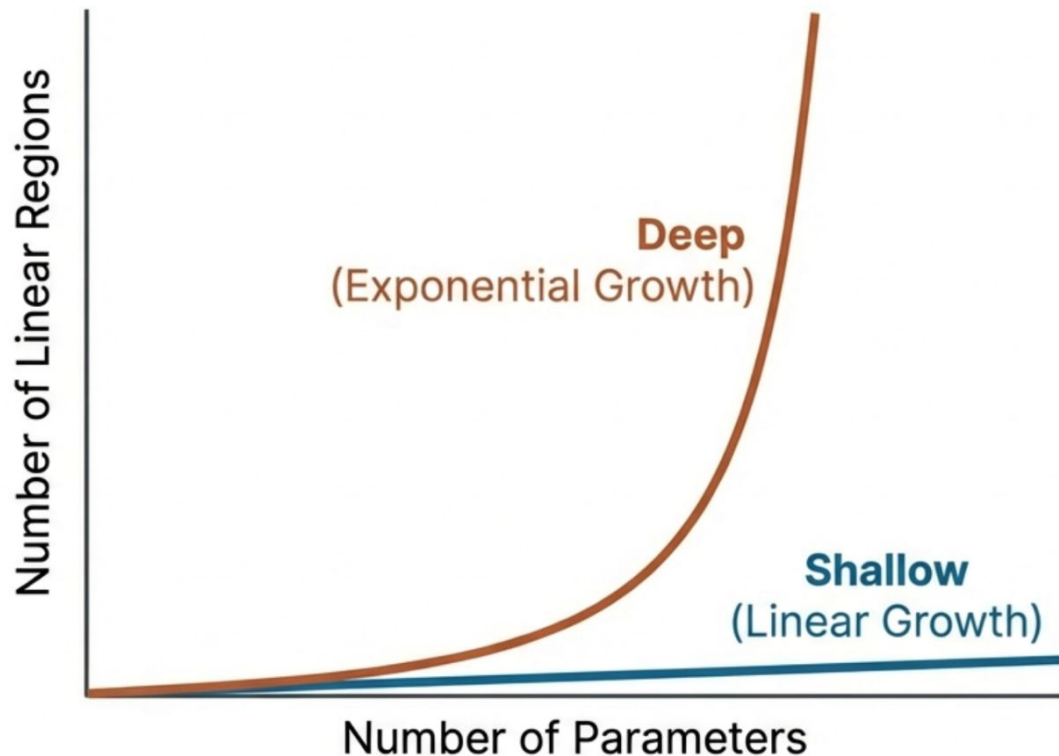


$$y = 1 \cdot \text{ReLU}(x) - 1 \cdot \text{ReLU}(-x)$$





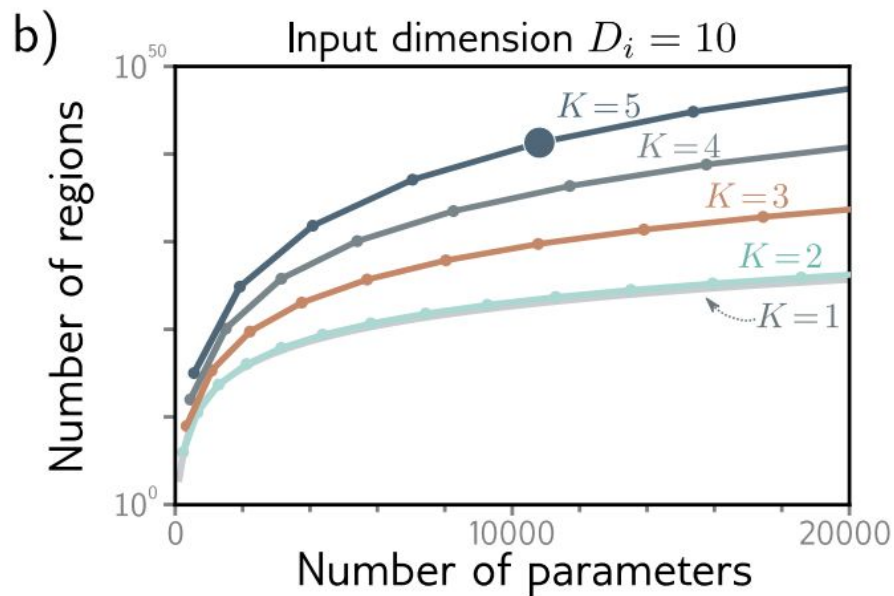
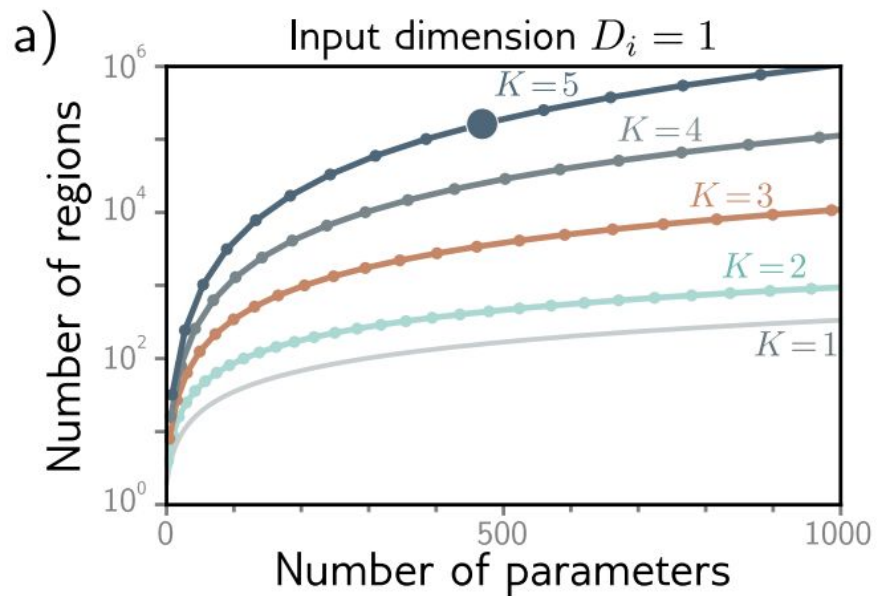
From Shallow to Deep Networks



Example:

A 5-layer network with 10 units/layer has only **471 parameters**. But it produces **161,051 linear regions**.

A shallow network with 471 parameters creates vastly fewer regions.

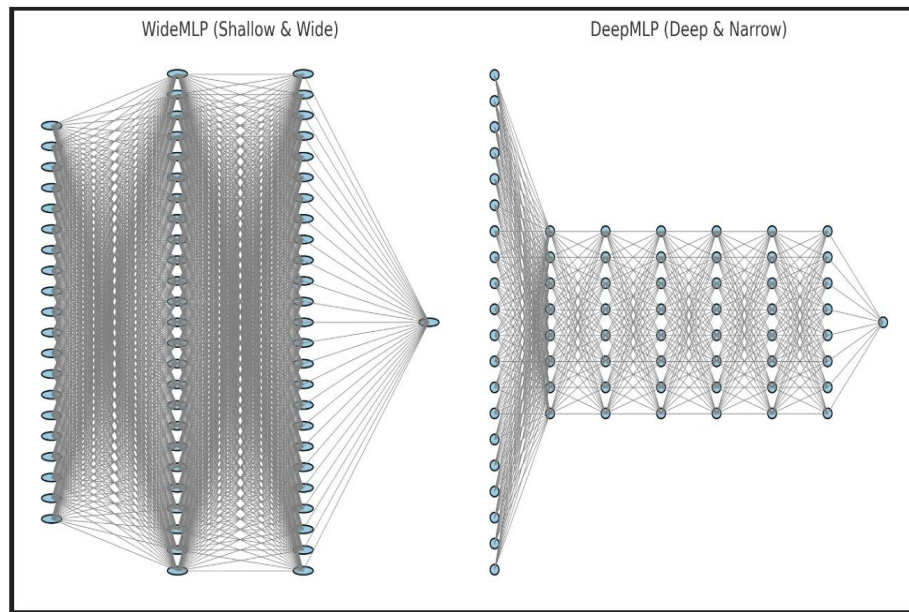


Role of Width (D)

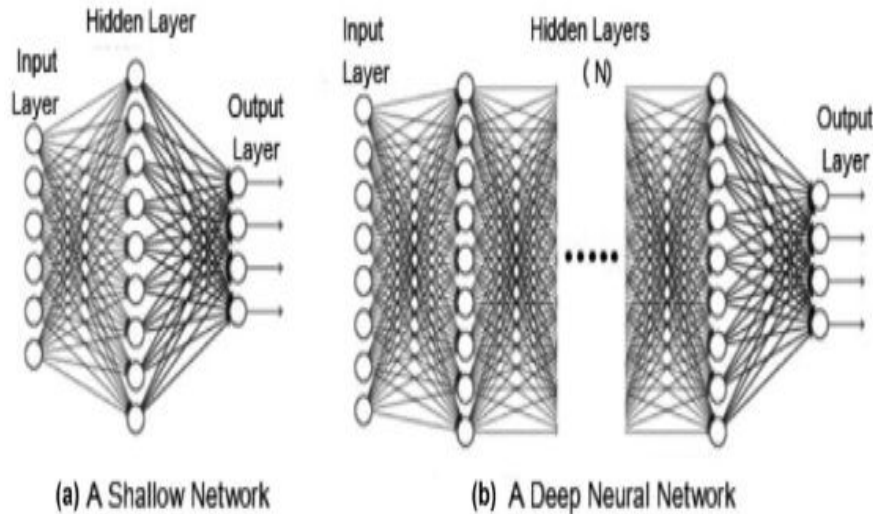
Width = Memory/Capacity

Adding neurons to a layer (increasing width) allows the network to capture more fine-grained details and variations in the input data.

Wide networks perform well on problems with less data or that are not overly complex (e.g., simple classification or regression), where a single or few wide layers can detect necessary patterns



Role of Depth (K)



Depth = Abstraction

Depth allows for compositional efficiency. A deep network can represent a function with $O(N)$ parameters that a shallow network would need $O(2N)$ parameters to represent.

Analogy: It is easier to describe a face as (Eyes + Nose + Mouth) than to describe every pixel coordinate individually.

Depth vs. Width Trade-off

Go Deeper (Modern Trend)

Better parameter efficiency. Captures hierarchical structure. Harder to train (Gradient issues).

Go Wider

Easier to optimize (smoother landscape). easier to parallelize (GPU friendly). Risk of overfitting.

Neural network **architectures** are primarily characterized by:

Depth → number of layers

Width → number of neurons per layer

Both influence model capacity, **expressiveness**, and computational **cost** — but in different ways.

Number of regions vs parameters

A shallow network with one input, one output, and $D > 2$ hidden units can create up to $D + 1$ linear regions and is defined by $3D + 1$ parameters. A deep network with one input, one output, and K layers of $D > 2$ hidden units can create a function with up to $(D + 1)^K$ linear regions using $3D + 1 + (K - 1)D(D + 1)$ parameters.

Actually it is..

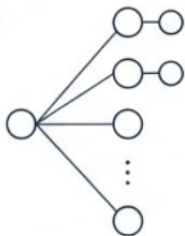
$$N_r = \left(\frac{D}{D_i} + 1 \right)^{D_i(K-1)} \cdot \sum_{j=0}^{D_i} \binom{D}{j}.$$

But how $3D + (K - 1) D (D - 1) + 1$ parameters?

Input Layer

Assuming 1 input dimension.

D weights +
 D biases = $2D$

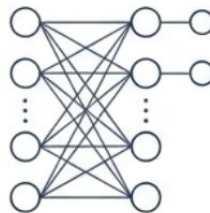


Hidden Layers

There are $K - 1$ connections between hidden layers.

Each connection:
 $D \times D$ weights +
 D biases = $D(D + 1)$.

Total for Hidden: $(K - 1)D(D + 1)$.



Output Layer

D weights + 1 bias = $D + 1$



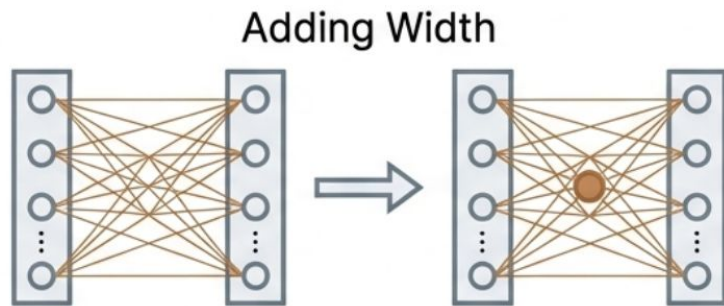
$$2D + (K - 1)D(D + 1) + D + 1$$



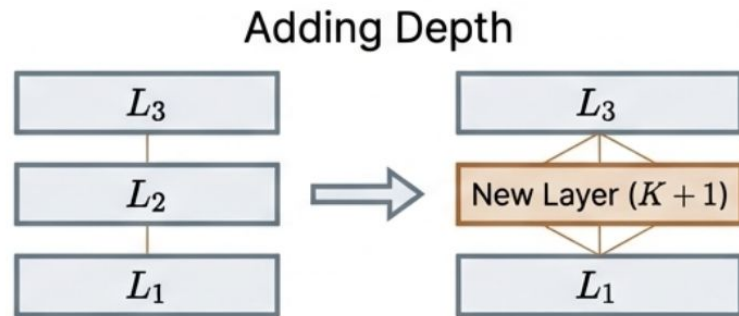
$$3D + 1 + (K - 1)D(D + 1)$$

Problem 4.6

Which is more expensive (adds more weights): adding 1 layer or 1 hidden unit?



Quadratic Growth ($O(D^2)$). Expensive.



Linear Growth ($O(K)$ once). Cheaper.

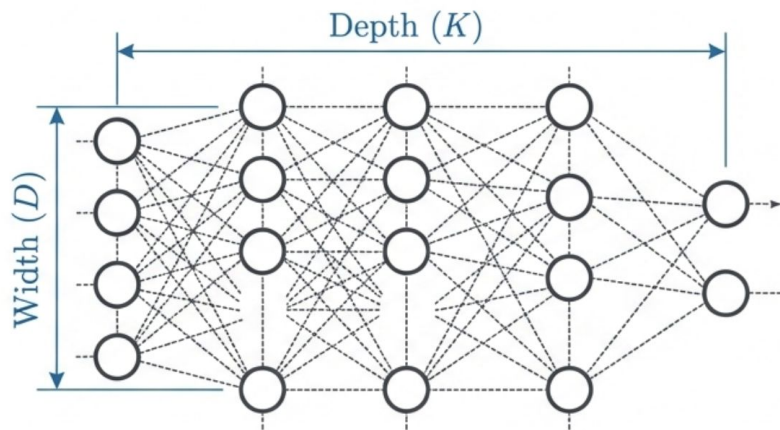
Conclusion: Increasing width is generally more expensive in parameter count than increasing depth.

Problem 4.11: The Showdown - Shallow vs. Deep

Compare Network A ($D = 95$, Shallow) vs. Network B ($D = 5$, $K = 10$, Deep)

Parameter	Network A (Shallow)	Network B (Deep)	Verdict
Cost	$3(95) + 1 = 286$	$3(5) + 1 + (9)(5)(6) = 286$	SAME COST
Linear Regions (Expressivity)	$95 + 1 = 96$	$\approx 2^{10}$ to 5^{10} (Millions)	DEEP WINS EXPONENTIALLY
Execution Speed	Parallelizable (One Matrix)	Sequential (10 Steps)	SHALLOW WINS

Designing the architecture



Hyperparameters

- Chosen *before* learning.
- Depth (K), Width (D).
- Defines the “Family of Functions”.

Parameters

- Learned *during* training.
- Weights (Ω), Biases (β).
- Selects the specific function.

Let's Play..



Epoch
000,000

Learning rate

0.03

Activation

ReLU

Regularization

L1

Regularization rate

0

Problem type

Classification

DATA

Which dataset do you want to use?



Ratio of training to test data: 50%

Noise: 15

Batch size: 11

REGENERATE

FEATURES

Which properties do you want to feed in?



+ - 3 HIDDEN LAYERS

+ -

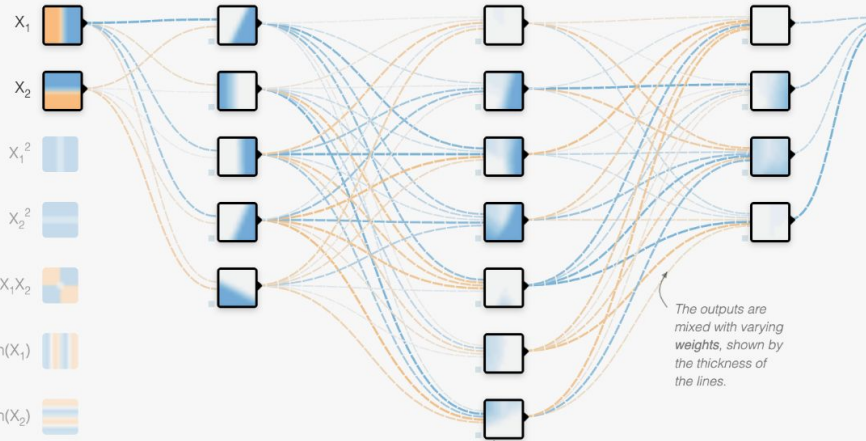
5 neurons

+ -

7 neurons

+ -

4 neurons



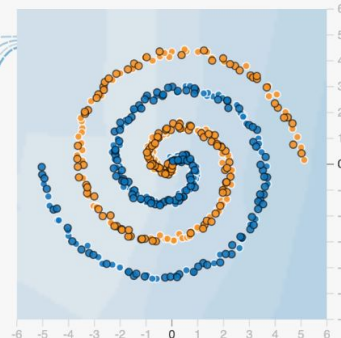
The outputs are mixed with varying weights, shown by the thickness of the lines.

This is the output from one neuron. Hover to see it larger.

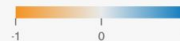
OUTPUT

Test loss 0.514

Training loss 0.526



Colors shows data, neuron and weight values.

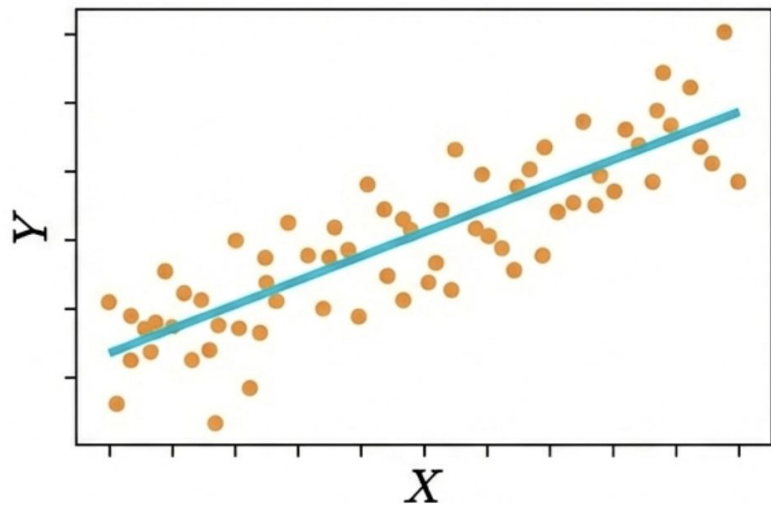


☒ Show test data

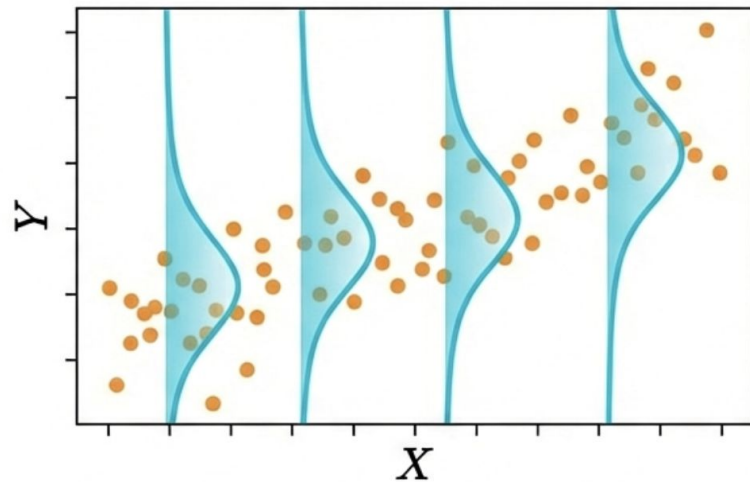
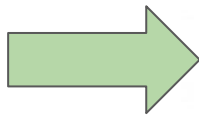
☐ Discretize output

Unifying the Loss functions

Perspective Shift : Predicting distributions not values



Model predicts a single value y .



Model predicts parameters θ of a distribution $Pr(y|x)$.

The network outputs mean μ and variance σ^2 .

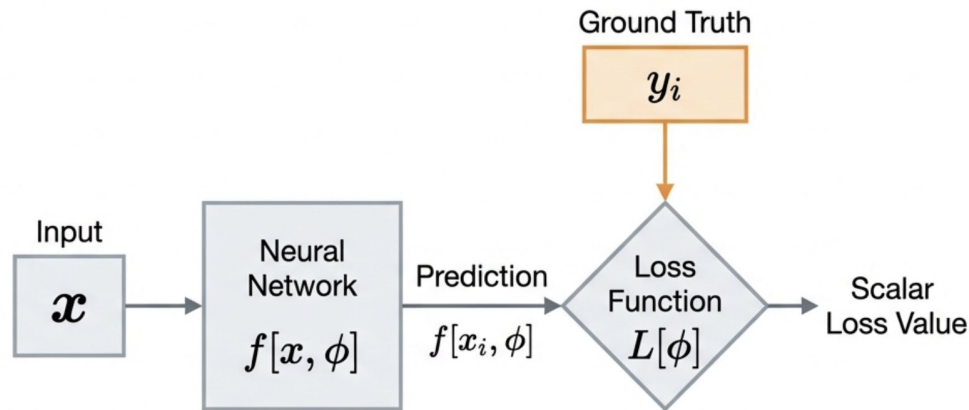
Understanding Loss

The Goal: Find parameters ϕ that map inputs to outputs as closely as possible.

The Definition:
A Loss Function $L[\phi]$ quantifies the mismatch.

The Objective:

$$\hat{\phi} = \operatorname{argmin}_{\phi} L[\phi]$$

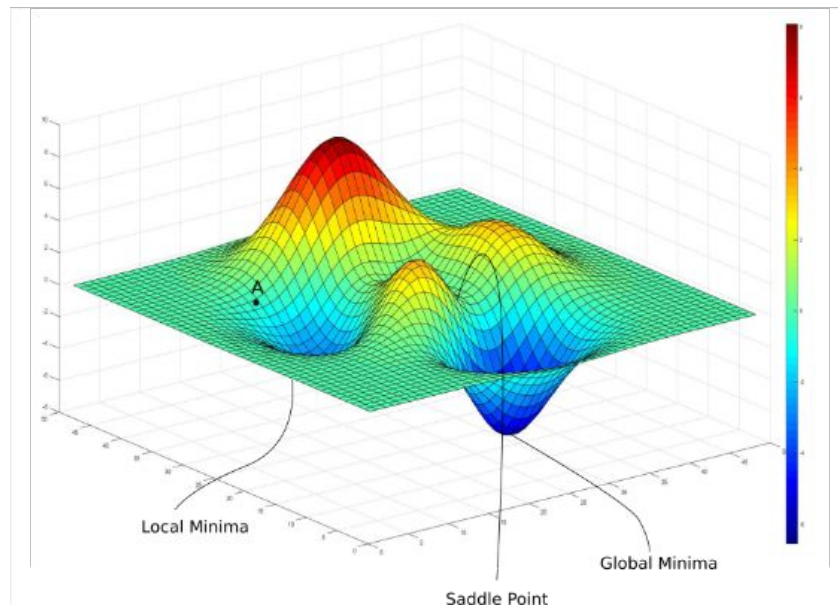


Visualizing the Loss Landscape

Imagine the Loss Function J as a geographical landscape (valleys and hills).

The "altitude" is the Error.

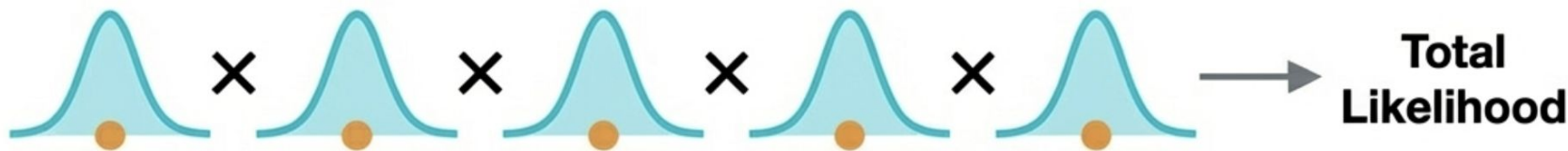
Our goal is to reach the lowest point (Global Minimum).



Maximum Likelihood

If we assume training data are Independent and Identically Distributed (i.i.d.), the probability of the entire dataset is the product of the individual probabilities.

$$Pr(\text{Dataset}|\phi) = \prod_{i=1}^I Pr(y_i | f[x_i, \phi])$$



We search for parameters $\hat{\phi}$ that maximize this product.

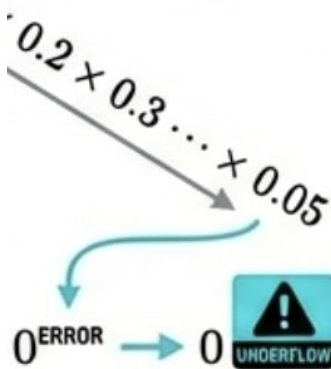
$$\hat{\phi} = \operatorname{argmax}_{\phi} \prod_{i=1}^I Pr(y_i | f[x_i, \phi])$$

From Likelihood to NLL and why ?

Why not maximize likelihood directly ?

Because probability values are small !!

Multiplying such small numbers repeatedly leads to floating point underflow issue.



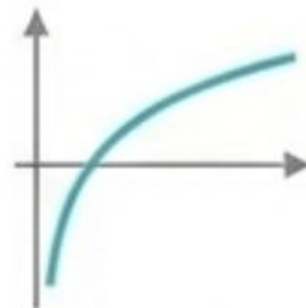
So what do we do ?

Negative Log Loss

Logarithms turn products into
 $\log(a \cdot b) = \log(a) + \log(b)$.

$$\prod_{i=1}^{\log(\cdot)} \rightarrow \sum_{i=1}$$

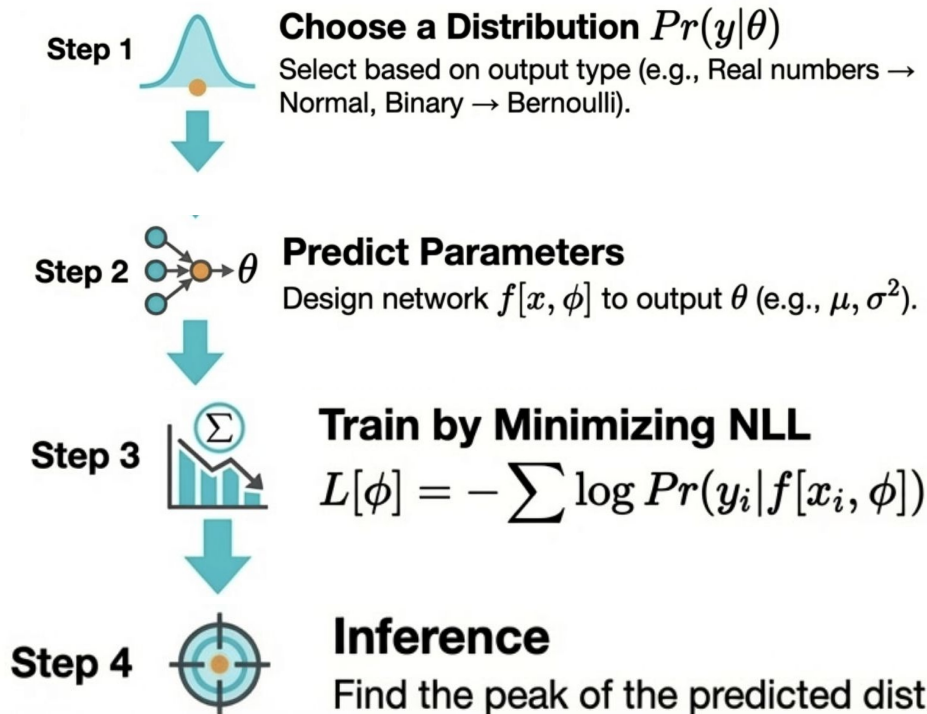
Log is monotonic: Maximizing
Likelihood is identical to
Maximizing $\log(\textit{Likelihood})$.



To create a **Minimization** problem
(Loss), we multiply by -1.

$$L[\phi] = - \sum_{i=1}^I \log [Pr(y_i | f[x_i, \phi])]$$

Universal Recipe for Loss Functions



**We do not
memorize loss
functions.
We derive them
using this 4-step
framework.**

Case Study 1 : Binary Classification

Step 1: The Distribution

Classes $y \in \{0, 1\}$.

Use **Bernoulli Distribution**.

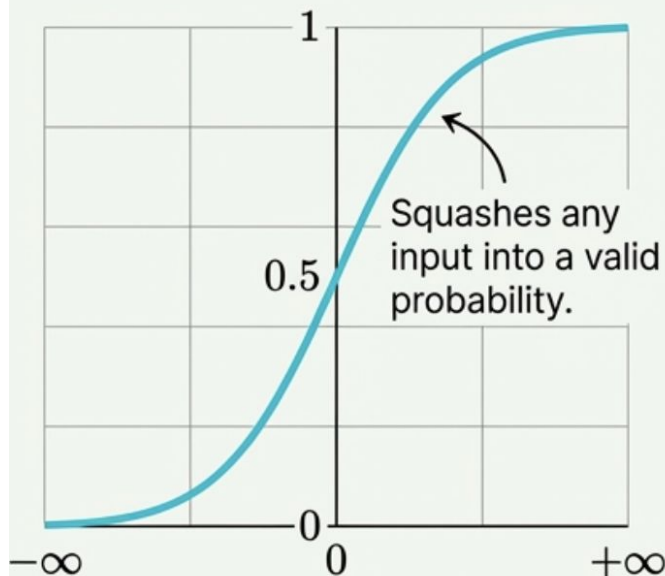
$$Pr(y|\lambda) = (1 - \lambda)^{1-y} \cdot \lambda^y$$

Step 2: The Parameter

Network predicts λ
(probability $y = 1$).

Constraint: $0 \leq \lambda \leq 1$.

The Tool: Sigmoid



$$\lambda = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Deriving Binary Cross Entropy

Applying **Negative Log-Likelihood** to the **Bernoulli formula**:

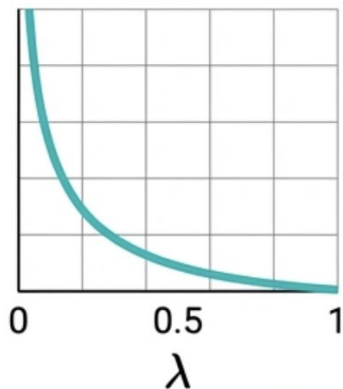
$$L_{BCE} = - \sum_{i=1}^I [y_i \log(\lambda_i) + (1 - y_i) \log(1 - \lambda_i)]$$

If $y = 1$:

Ground Truth is 1.

Loss term: $-\log(\lambda)$

Graph: Loss explodes
if prediction $\lambda \rightarrow 0$.

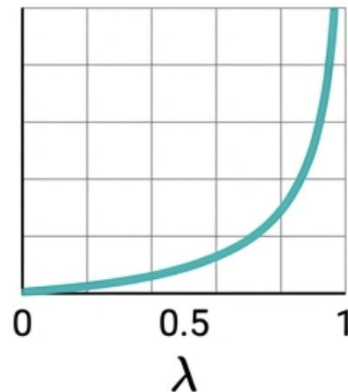


If $y = 0$:

Ground Truth is 0.

Loss term: $-\log(1 - \lambda)$

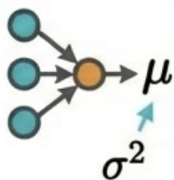
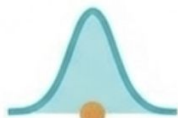
Graph: Loss explodes
if prediction $\lambda \rightarrow 1$.



Case Study 2 : Univariate Regression

The Recipe

- **Step 1:** Choose **Normal Distribution** (defined by μ, σ^2).
- **Step 2:** Network predicts mean μ . Variance σ^2 is assumed constant.



The Derivation

$$Pr(y|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y - \mu)^2}{2\sigma^2} \right] \quad (1)$$

Apply $-\log$

$$-\log Pr(y|\dots) = \frac{1}{2} \log(2\pi\sigma^2) + \frac{(y - \mu)^2}{2\sigma^2} \quad (2)$$

Since σ^2 is constant, we only minimize the varying part:

$$L = \sum_{i=1}^I (y_i - f[x_i, \phi])^2 \quad (3)$$

The Recipe is Universal !

Data Type	Visual Example	Required Distribution
Count Data (Integer ≥ 0)	 A digital counter displaying the number 123 and a parking meter with four groups of tally marks (three groups of three and one group of two).	Poisson Distribution
Directional ($-\pi$ to π)	 A compass rose with cardinal directions (N, S, E, W) and ordinal directions (NE, SE, SW, NW) labeled. A red arrow points towards the North-North-East.	Von Mises Distribution
Bounded Interval ($0 < y < 1$)	 A horizontal slider bar with a blue track and a white knob. The track is labeled 0 at the left end and 1 at the right end. The knob is positioned at approximately 0.7.	Beta Distribution